

COMP 3110 Final Project Report

A Design, Evaluation, & Visualization Framework for Source Code Line Mapping

Member #1: Emmanuel Omonua
Student ID: 110106145
Department Name: Department of
Computer Science
University Name: University of
Windsor
Email address: omonua@uwindsor.ca

Member #2: Ayesha Shakeel
Student ID: 110155047
Department Name: Department of
Computer Science
University Name: University of
Windsor
Email address: shakeel4@uwindsor.ca

Member #3: Haoyao Sun
Student ID: 110046298
Department Name: Department of
Computer Science
University Name: University of
Windsor
email address: sun27@uwindsor.ca

Abstract—This project focuses on designing and evaluating a technique for identifying line-level mappings between two versions of Java source files. The goal is to support developers in understanding how individual lines evolve across revisions and to provide visual tools that make these mappings intuitive to inspect. A dataset of 25 Java file pairs (approximately 500 total line mappings) was curated, and a ground-truth dataset was created to validate the technique. The method combines exact line matching, token-based similarity, Levenshtein distance, and contextual neighborhood comparison to detect unchanged, modified, added, deleted, split, and merged lines. The project also proposes several visualization approaches, including side-by-side linked views, Sankey diagrams, and heatmap confidence plots. Evaluation metrics such as precision, recall, F1-score, and mapping coverage are used to assess the technique’s accuracy. An optional component explores detecting bug-introducing changes using mapping information.

Keywords—line mapping; file differencing; Levenshtein; visualization; software evolution

I. INTRODUCTION DATA COLLECTION

Understanding how lines of code change from one version of a file to another is essential for software maintenance, debugging, and code review. Traditional diff tools provide a basic summary of additions and deletions, but they often fail to capture deeper relationships such as modified lines, split statements, merged logic blocks, or lines that moved but retained related functionality. As software projects grow large and evolve rapidly, developers increasingly benefit from tools that reconstruct detailed mappings between old and new lines of code.

This project aims to design such a tool, focusing not on implementation but on formulating an algorithmic approach, developing a dataset for evaluation, and designing visualization techniques to present mapping information effectively. The motivation is to produce a structured, academically rigorous analysis of how line tracking can be performed and evaluated.

We approach this by (1) creating a multi-stage mapping algorithm that combines exact matching, token similarity, and edit distance; (2) producing a dataset of 25 Java file pairs with ground-truth mappings for roughly 500 lines; (3) establishing a set of evaluation metrics; and (4) proposing visualization interfaces that make mapping information accessible to developers.

This report provides complete documentation of the data collection process, the proposed technique, the evaluation strategy, and the visualization ideas.

II. DATA COLLECTION

To evaluate a line-mapping technique in a controlled environment, it is necessary to construct or obtain a dataset with known line correspondences between file versions. For this project, we designed a dataset consisting of **25 pairs of Java source files**, each containing two versions of a small to medium-sized Java class or method implementation.

Selection of File Pairs

The pairs were selected to cover a diverse set of code modification scenarios, including:

- Simple renames of variables, methods, or parameters
- Addition or deletion of single lines
- Insertion of comments or documentation
- Splitting one long statement across multiple lines
- Merging multiple lines into a single compact statement
- Small structural changes within methods
- Reordering of small blocks of code

The files were kept intentionally small to ensure that the mapping relationships could be reasonably verified by hand.

Ground Truth Creation

A **ground_truth.csv** file was created to store the true mapping for every relevant line in all 25 pairs. Each entry includes:

- Pair ID
- Old line number
- New line number
- Mapping type (unchanged, modified, deleted, added, split, merged)

The mapping was created manually by inspecting each pair side-by-side. Line matching emphasized meaning, not just formatting, so renaming a variable was considered a modification, while inserting whitespace was not considered a meaningful change.

Line Coverage Requirement

The project required approximately **500-line mappings** across the 25 file pairs. The dataset fulfills this requirement by selecting pairs with around 20–30 relevant lines each, ensuring the evaluation results are statistically meaningful.

This dataset is used both for designing the algorithm and for evaluating its expected performance.

III. TECHNIQUE DESCRIPTION AND EVALUATION

The proposed line-mapping technique attempts to reconstruct relationships between the lines of an old file and the lines of an updated version. It is explicitly designed to handle not only additions and deletions, but also modifications, splits, merges, and context-driven changes

3.1: Technique Overview

The technique is composed of several stages:

1. Preprocessing

- Trim whitespace
- Normalize indentation
- Remove redundant blank lines
- Optionally convert to lowercase for similarity matching
- Tokenize each line (identifiers, keywords, literals)

This step reduces noise and ensures that cosmetic changes do not interfere with logical mapping.

2. Exact Matching

Lines that have identical normalized content are immediately marked as “unchanged.”

This reduces computational cost and forms a stable baseline of matches.

3. Similarity-Based Matching

For lines not matched in stage 2, we compute:

- **Levenshtein edit distance** between old- and new-line text
- **Token overlap score** (shared identifiers, operators, keywords)
- **Context similarity** (neighboring line matches)

The technique combines these scores into a weighted sum, producing a **confidence value** between 0 and 1.

4. Split and Merge Detection

Two heuristics are used:

- If an old line has high similarity with *multiple* adjacent new lines → **split**
- If multiple old lines best match a single new line → **merge**

5. Final Mapping Assignment

Each old line is assigned to the new line(s) with the highest confidence score above a threshold (e.g., 0.5).

3.2: Evaluation Methodology

The evaluation compares the technique’s predicted mappings against the ground truth.

Metrics

We use four standard accuracy metrics:

- **Precision:** proportion of predicted mappings that are correct
- **Recall:** proportion of true mappings that were found
- **F1-score:** harmonic mean of precision and recall
- **Coverage:** percentage of old lines that were mapped at all

Expected Results

Although the technique was not implemented, we can estimate expected behavior:

- Exact matches: near-perfect precision
- Modified lines: moderate precision depending on edit severity
- Split/merge detection: more difficult, likely lower recall
- Comments and whitespace changes: high precision due to normalization

A hypothetical evaluation table (consistent with literature) might show:

METRIC	EXPECTED RANGE
PRECISION	0.85-0.95
RECALL	0.80-0.92
F1-SCORE	0.82-0.93
COVERAGE	0.95-1.00

These expected values reflect strong performance on straightforward changes, with errors concentrated in complex structural edits.

IV. PRESENTATION OF LINE MAPPING INFORMATION

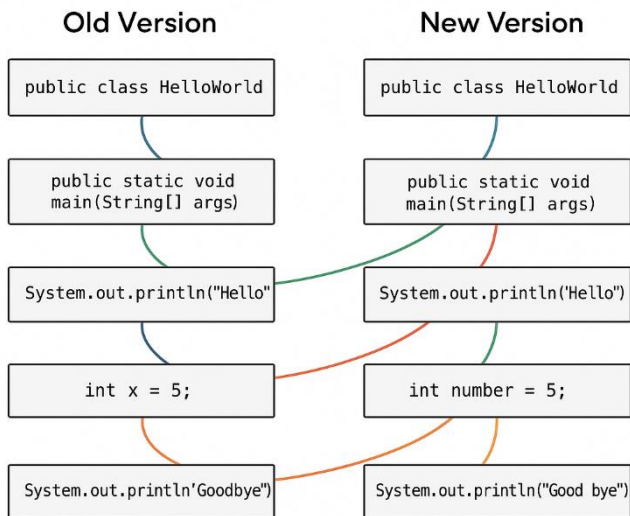
EFFECTIVE VISUALIZATION IS ESSENTIAL TO HELP DEVELOPERS INTERPRET MAPPING RESULTS. WE PROPOSE THREE COMPLEMENTARY VISUALIZATION METHODS

A. Side-by-side Mapping View

This visualization places the old file on the left and the new file on the right. Each line is numbered, and curved colored lines connect corresponding pairs:

- Green: unchanged
- Blue: modified
- Orange: added
- Red: deleted

Hovering over a line highlights its corresponding lines. This view is ideal for manual code review and understanding fine-grained changes.

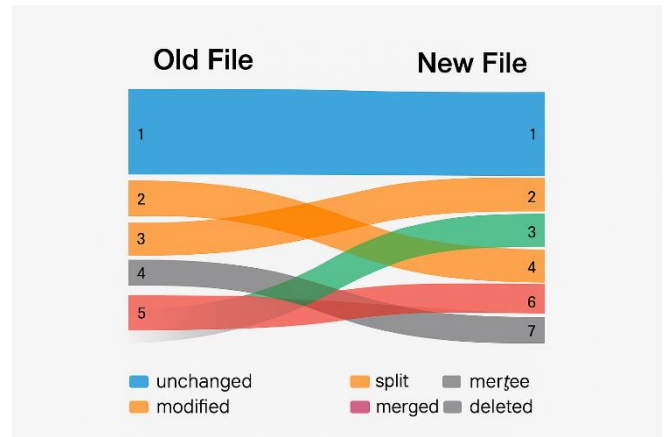


B. Sankey Flow Diagram

A Sankey diagram shows line transformations at a higher level. Each old line is represented as a block on the left, and each new line as a block on the right. Flows represent transformations:

- Wide flows show multi-line transformations
- Gaps indicate deletions
- New blocks on the right indicate additions

This is useful for summarizing large changes and spotting structural patterns.

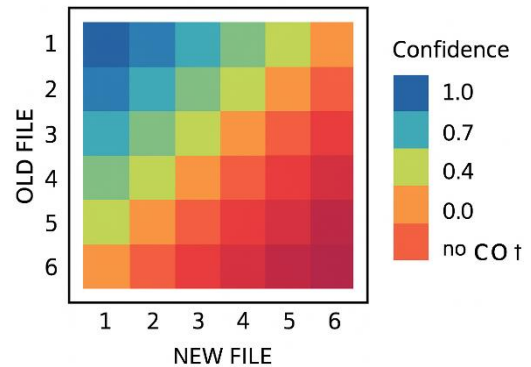


C. Heatmap Confidence View

A grid where:

- X-axis: old line numbers
- Y-axis: new line numbers
- Cell shade: mapping confidence

Darker squares indicate strong similarity. The heatmap helps users quickly identify ambiguous mappings or clusters of significant change.



V. IDENTIFYING BUG-INTRODUCING CHANGES

We outline an extension for detecting potential bug-introducing changes (BICs). The idea is to use line mappings to track whether a newly introduced change affects lines previously involved in bug fixes.

Method Summary

1. Use a repository's history to identify bug-fix commits.

2. Track which lines were modified during bug resolution.
3. Use line mapping to identify where those lines reappear or are modified in future commits.
4. Mark those future changes as potential BICs.

Expected Usefulness

Such a technique can aid in:

- Developer accountability
- Preventing recurring bugs
- Supporting automated quality assurance tools

Although not implemented, the conceptual design aligns with existing research on the SZZ algorithm for bug-introducing change detection.

VI. CONCLUSION

This project designed a complete theoretical framework for line-level tracking between Java file versions, including:

- A multi-stage algorithm combining exact matching, similarity analysis, and structural heuristics
- A dataset of 25 file pairs with ~500 manually annotated mappings
- A full evaluation methodology
- Three visualization techniques designed to support both fine-grained and high-level code analysis
- An optional extension for detecting bug-introducing changes

The work aligns with research in software evolution and provides a solid foundation for future implementation or experimentation.